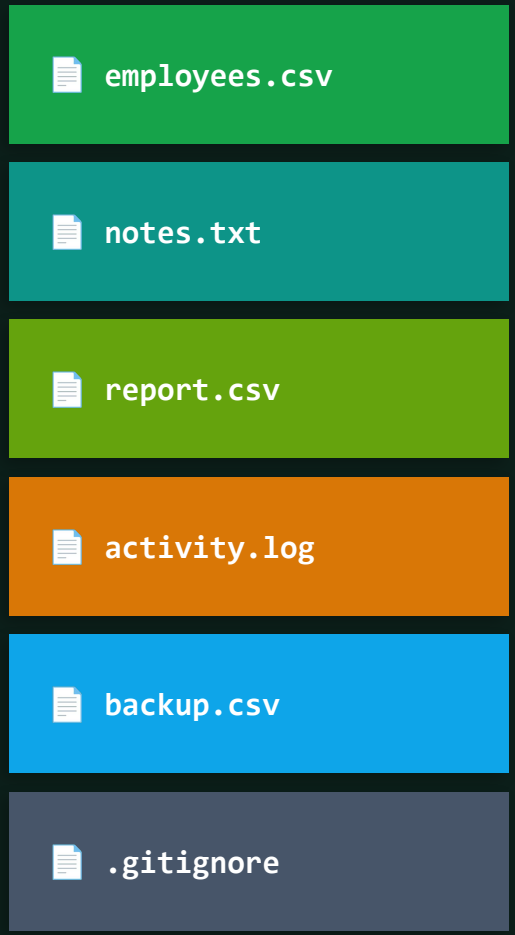
DAY 7

File Handling + CSV Data Management

Read, write, organise & manage real data with Python files and CSV databases



M1 File Fundamentals | M2 Read & Write | M3 Exception Handling | M4 CSV Module | M5 Data Management | M6 Pro File Structure

MODULE 1

File Handling Fundamentals

What are files, why they matter, and how Python opens them

📁 What is File Handling?

File Handling = Reading from and writing to files stored on disk. Without it, all your program's data disappears the moment you close it.

🧠 Analogy — Why File Handling Exists

Imagine your Python program is a whiteboard. When you close it, the whiteboard is wiped clean. Files are like notebooks — they store data PERMANENTLY on disk so you can come back to it tomorrow.

📝 Text Files (.txt)

Stores plain human-readable text.
Note-taking, logs, config files, diary entries.

```
Examples:
notes.txt
logs.txt
config.txt
```

📊 CSV Files (.csv)

Comma-Separated Values. Like a spreadsheet.
Employee records, products, student data.

```
Examples:
employees.csv
students.csv
report.csv
```

⚙️ Binary Files

Images, audio, executables. Not human-readable.
Needs special libraries to handle.

```
Examples:
photo.jpg
audio.mp3
program.exe
```

File Modes — How to Open a File

Every time you open a file, you must tell Python WHAT you want to do with it. The mode controls read/write permissions.

"r"

Read

Opens file for READING only.
Default mode. Raises FileNotFoundError if file does not exist.
File pointer starts at the beginning.

✅ Safe — cannot accidentally overwrite data

"w"

Write

Opens file for WRITING. Creates file if not found.
⚠️ DESTROYS existing content — starts fresh!
Use carefully. Great for creating new files.

⚠️ Danger — overwrites existing content!

"a"

Append

Opens file for APPENDING. Creates if not found.
Adds new content at the END. Never deletes old data.
Perfect for log files and ongoing records.

✅ Safe — adds to end, never deletes

"x"

Create

Creates a brand new file.
Raises FileExistsError if file already exists.
Use when you want to guarantee a fresh file.

✅ Safe — refuses to overwrite

"r+"

Read + Write

Opens for reading AND writing.
File must already exist. Does NOT truncate.
Used for in-place updates to existing files.

⚠️ File must exist; can overwrite data

"a+"

Append + Read

Opens for reading AND appending.
Reads from start, writes always at end.
Good for reading logs while also adding to them.

✅ Reads all, writes at end only

File Paths — Relative vs Absolute

A file path tells Python WHERE to find or save a file. Getting this wrong is one of the most common beginner mistakes.

Relative Path

Relative to where your Python file is running.
Shorter, more portable across computers.

```
# Your project folder:
# my_project/
# ├── main.py      ← running from here
# └── data/
#     └── notes.txt

# Relative paths:
open("notes.txt")        # same folder
open("data/notes.txt")   # subfolder
open("../other.txt")     # one level up
```

Absolute Path

Full path from the root of the drive.
Always works regardless of current folder.

```
# Windows absolute path:
open("C:/Users/Alice/notes.txt")

# Mac/Linux absolute path:
open("/home/alice/notes.txt")

# ── Best Practice (using os module) ──
import os
base = os.path.dirname(__file__)
path = os.path.join(base,"data","notes.txt")
open(path)   # always works!
```

MODULE 2

Reading & Writing Files

open(), read, write, append — with the context manager

📖 Reading Files — 3 Ways to Read

Python gives you 3 reading functions. Each is designed for a different situation. Use the right one for your task.

📄 .read()

Reads the ENTIRE file content as one big string.
Best for small files (config, settings, short text).

```
with open("notes.txt", "r") as f:
    content = f.read()
print(content)
# Hello World
# This is line 2
# This is line 3
```

⚠️ Avoid for large files — loads everything into memory at once

➡ .readline()

Reads ONE line at a time. Each call returns the next line.
Good for processing line by line without loading everything.

```
with open("notes.txt", "r") as f:
    line1 = f.readline()  # "Hello
World\n"
    line2 = f.readline()  # "This is
line 2\n"
print(line1.strip())   # Hello World
```

✅ Great for large files — reads one line at a time

📋 .readlines()

Reads ALL lines and returns them as a Python LIST.
Each item in the list is one line of the file.

```
with open("notes.txt", "r") as f:
    lines = f.readlines()
print(lines)
# ['Hello World\n','line 2\n',...]
print(len(lines))  # 3 (number of
lines)
```

✅ Best for iterating over every line in the file

✏ Writing Files & the Context Manager

```
# ── METHOD 1: Basic write() ──────────────────────────────────────────
f = open("notes.txt", "w")   # opens file
f.write("Hello World\n")     # write line 1
f.write("Python is great\n") # write line 2
f.close()                    # ← MUST close! (or data may not be saved)

# ── METHOD 2: writelines() for multiple lines ────────────────────────
lines = ["Alice\n", "Bob\n", "Charlie\n"]
f = open("names.txt", "w")
f.writelines(lines)          # writes all at once
f.close()

# ── METHOD 3: Context Manager — THE BEST WAY ✅ ──────────────────────
# Automatically closes file even if an error occurs inside the block
with open("notes.txt", "w") as f:
    f.write("Line 1\n")
    f.write("Line 2\n")
    f.write("Line 3\n")
# ← file closes automatically here — no f.close() needed!

# ── APPEND mode — adding without deleting ────────────────────────────
with open("log.txt", "a") as f:
    f.write("2024-01-15: User logged in\n")   # adds at END
    f.write("2024-01-15: File opened\n")      # original content stays!
```

💡 **Why use `with open() as f:` — the Context Manager?**

🛠️ Hands-On: Student Notes System + Daily Task Logger

```python
# ── Student Notes System ──────────────────────────────────────────────
def save_note(subject, content):
    with open(f"{subject}_notes.txt", "a") as f:
        f.write(f"--- Note ---\n{content}\n\n")
    print(f"Note saved to {subject}_notes.txt")

def view_notes(subject):
    try:
        with open(f"{subject}_notes.txt", "r") as f:
            print(f.read())
    except FileNotFoundError:
        print(f"No notes found for {subject}")

# Usage
save_note("Python", "Variables store data in memory")
save_note("Python", "Functions are reusable code blocks")
view_notes("Python")
```

```python
# ── Daily Task Logger (Append mode keeps full history) ────────────────
from datetime import datetime

def log_task(task, status="DONE"):
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M")
    with open("daily_log.txt", "a") as f:
        f.write(f"[{timestamp}] [{status}] {task}\n")
    print(f"Logged: {task}")
```

MODULE 3

Exception Handling with Files

Graceful error handling — never let file errors crash your app

⚠️ File Exceptions & Safe Error Handling

Files can go wrong in many ways. Professional code anticipates these failures and handles them gracefully instead of crashing.

🔍 FileNotFoundError

File path doesn't exist. Most common error.
Fix: check path, use os.path.exists() first.

🔒 PermissionError

No read/write permission on that file.
Fix: check OS permissions, use try/except.

📁 IsADirectoryError

You gave a folder path instead of a file.
Fix: verify the path points to a file.

💾 IOError

Low-level input/output error (disk full etc).
Fix: check disk space, try/except + log.

```
# ── Safe File Reader — handles ALL error cases ──────────

def safe_read(filepath):
    try:
        with open(filepath, "r") as f:
            content = f.read()
            print(content)
            return content

    except FileNotFoundError:
        print(f"❌ File not found: {filepath}")
        print("   Check the path and try again.")
        return None

    except PermissionError:
        print(f"🔒 No permission to read: {filepath}")
        return None

    except IsADirectoryError:
        print(f"📁 That's a folder, not a file!")
        return None

    except IOError as e:
        print(f"💾 IO Error occurred: {e}")
        return None

    finally:
```

MODULE 4

CSV Fundamentals

Comma-Separated Values — Python's built-in spreadsheet

What is CSV? Structure & Real-World Uses

CSV (Comma-Separated Values) is a universal data format. Excel, Google Sheets, databases — they all use CSV for import/export.

employees.csv — raw file content

```
id,name,department,salary,city
1,Alice Johnson,Engineering,85000,Mumbai
2,Bob Smith,Marketing,62000,Delhi
3,Charlie Brown,HR,55000,Bangalore
4,Diana Prince,Engineering,92000,Mumbai
5,Eve Wilson,Marketing,67000,Pune
```

Same data — how it looks in Excel / Google Sheets

id	name	department	salary	city
1	Alice Johnson	Engineering	85000	Mumbai
2	Bob Smith	Marketing	62000	Delhi
3	Charlie Brown	HR	55000	Bangalore

Employee Records

HR departments export staff data to CSV for payroll & analytics

Inventory Tracking

Products, quantities, prices stored in CSV for easy editing

Student Results

Marks, grades, attendance — CSV lets teachers share data easily

Expense Reports

Finance teams use CSV to import transactions into accounting software

API Data Export

Web APIs offer CSV download for reports, dashboards, and charts

csv.reader() and csv.writer()

Python's built-in csv module handles all the tricky parts of CSV (commas inside values, quotes, encoding) automatically.

```
import csv

# ═══ READING CSV with csv.reader() ═══════════════════════════════════
with open("employees.csv", "r", newline="") as f:
    reader = csv.reader(f)          # creates a reader object
    header = next(reader)           # skip first row (header)
    print("Columns:", header)       # ['id','name','department','salary','city']

    for row in reader:              # each row is a LIST
        print(f"Name: {row[1]}  Dept: {row[2]}  Salary: ₹{row[3]}")

# Output:
# Name: Alice Johnson  Dept: Engineering  Salary: ₹85000
# Name: Bob Smith      Dept: Marketing    Salary: ₹62000

# ═══ WRITING CSV with csv.writer() ═══════════════════════════════════
students = [
    ["id", "name",    "marks", "grade"],  # header row
    [1,    "Alice",   92,      "A"     ],
    [2,    "Bob",     78,      "B"     ],
    [3,    "Charlie", 85,      "A"     ],
]

with open("students.csv", "w", newline="") as f:
```

newline=""
Always add this to open() when using the csv

next(reader
)
Call this once to skip the header row before iterating

writerows()
Writes a LIST of lists at once — faster than calling

🗂️ DictReader & DictWriter — The Professional Way

DictReader and DictWriter use column NAMES instead of index numbers. Much cleaner — use these in real projects.

```
import csv

# ═══ DictReader — reads each row as a DICTIONARY ═══════════════════════
with open("employees.csv", "r", newline="") as f:
    reader = csv.DictReader(f)      # column names become keys automatically!

    for row in reader:
        # Access by column NAME (not index number)
        print(f"Name: {row['name']}  Dept: {row['department']}  Salary: {row['salary']}")
        # Much clearer than: row[1], row[2], row[3]

# ═══ DictWriter — writes dictionaries to CSV ═══════════════════════════
employees = [
    {"id":1, "name":"Alice",   "department":"Engineering", "salary":85000},
    {"id":2, "name":"Bob",     "department":"Marketing",   "salary":62000},
    {"id":3, "name":"Charlie", "department":"HR",          "salary":55000},
]

fieldnames = ["id", "name", "department", "salary"]  # defines column ORDER

with open("output.csv", "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=fieldnames)
    writer.writeheader()            # writes: id,name,department,salary
```

❌ reader (hard to read): row[3] — what's index

✅ DictReader (clear): row['salary'] — obvious!

MODULE 5

Data Management Using CSV

Create · Search · Update · Delete · Filter · Report

⚙️ CSV Database — CRUD Operations

CSV files can be used as a lightweight database. Create, Read, Update and Delete — the 4 operations every data system needs.

```
import csv, os

FILENAME = "employees.csv"
FIELDS   = ["id","name","department","salary","city"]

# ── CREATE: Add new employee ──────────────────────────────────────
def add_employee(emp_dict):
    file_exists = os.path.exists(FILENAME)
    with open(FILENAME, "a", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=FIELDS)
        if not file_exists:
            writer.writeheader()          # write header only if new file
        writer.writerow(emp_dict)
    print(f"Added: {emp_dict['name']}")

# ── READ: View all employees ──────────────────────────────────────
def view_all():
    with open(FILENAME, "r", newline="") as f:
        reader = csv.DictReader(f)
        for row in reader:
            print(f"[{row['id']}] {row['name']} | {row['department']} | ₹{row['salary']}")

# ── SEARCH: Find by name or department ────────────────────────────
def search(keyword):
    with open(FILENAME, "r", newline="") as f:
```

📈 Filtering Data & Generating Reports

```
import csv
from collections import defaultdict

# ── FILTER: Department-wise employees ──────────────────────────
def filter_by_department(dept):
    with open("employees.csv","r",newline="") as f:
        reader = csv.DictReader(f)
        return [r for r in reader if r["department"].lower() == dept.lower()]

eng_team = filter_by_department("Engineering")
print(f"Engineering team: {len(eng_team)} employees")

# ── REPORT: Department-wise salary summary ─────────────────────
def department_report():
    dept_data = defaultdict(lambda: {"count":0,"total_salary":0})
    with open("employees.csv","r",newline="") as f:
        for row in csv.DictReader(f):
            dept = row["department"]
            dept_data[dept]["count"]        += 1
            dept_data[dept]["total_salary"] += int(row["salary"])

    print("\n=== DEPARTMENT REPORT ===")
    print(f"{'Department':<20} {'Count':>6} {'Avg Salary':>12}")
    print("-" * 42)
    for dept, data in sorted(dept_data.items()):
        avg = data["total_salary"] // data["count"]
```

Sample Output:

```
=== DEPARTMENT REPORT ===
Department           Count   Avg Salary
Engineering              2     ₹88,500
HR                       1     ₹55,000
```

MODULE 6

Professional File Organisation

Folder structure, naming conventions & data best practices

📂 Professional Project File Structure

Separating source code from data files, reports, and logs makes your project maintainable, testable, and ready for teams.

```
employee-log-system/          ← Root project folder
│
├── src/                      ← All Python source code
│   ├── main.py               ← Entry point (start here)
│   ├── employee.py           ← Employee class & CRUD logic
│   ├── storage.py            ← All CSV read/write functions
│   ├── reports.py            ← Report generation
│   └── utils.py              ← Helper functions (validation etc)
│
├── data/                     ← All data files
│   ├── employees.csv         ← Main employee database
│   └── attendance.csv        ← Attendance records
│
├── reports/                  ← Generated report outputs
│   ├── dept_summary.csv      ← Department-wise export
│   └── monthly_report.csv    ← Monthly attendance report
│
```

Separate code and data
Never hardcode file paths inside src/. Keep data/ separate from Python files.

Meaningful file names
employees.csv not e.csv. dept_report_jan2024.csv not r1.csv. Future-you says thanks.

Daily log files
Name logs by date: activity_2024-01-15.log — never overwrite, always append.

Validate before writing
Check data types and ranges before saving. One bad row can corrupt analytics.

Automatic backups
Before every bulk update, copy current CSV to backups/ with today's date in filename.

Use os.path.join()
Never string-concatenate paths. os.path.join('data','emp.csv') works on all OS.

MINI PROJECT

Employee Log System

File-based employee management using TXT + CSV | Basic → Intermediate → Advanced

🏗️ Employee Log System — Feature Levels

🟢 BASIC

- ▸ Add Employee (ID, name, dept, salary, city)
- ▸ View All Employees (formatted table)
- ▸ Search Employee by name or dept
- ▸ Delete Employee by ID
- ▸ Save & Load data from employees.csv

🟡 INTERMEDIATE

- ▸ Update Employee Details (salary, dept)
- ▸ Department-wise Filtering & Stats
- ▸ Attendance Logging (present/absent)
- ▸ CSV Export of any filtered data
- ▸ Input validation & error handling

🔴 ADVANCED

- ▸ Activity Logs (who did what, when)
- ▸ Monthly Attendance Report CSV
- ▸ Backup system (auto-backup on modify)
- ▸ Report Generation (dept summary)
- ▸ Full menu-driven professional app

💻 Employee Log System — Core Code

```
# employee.py — Employee class
class Employee:
    def __init__(self, emp_id, name, department, salary, city):
        self.emp_id     = emp_id
        self.name       = name
        self.department = department
        self.salary     = float(salary)
        self.city       = city

    def to_dict(self):
        return {"id":self.emp_id,"name":self.name,"department":self.department,
                "salary":self.salary,"city":self.city}

# storage.py — CSV CRUD
import csv, os, shutil
from datetime import datetime

FILENAME = "data/employees.csv"
FIELDS   = ["id","name","department","salary","city"]

def load_all():
    if not os.path.exists(FILENAME): return []
    with open(FILENAME,"r",newline="") as f:
        return list(csv.DictReader(f))

def save_all(records):
    # backup before overwriting
```

```
# main.py — Menu-driven interface
def main():
```

Advanced: Attendance Log + Backup System

```
# ── Attendance Logging ──────────────────────────────────────────────
import csv
from datetime import datetime

def log_attendance(emp_id, status):  # status = "Present" or "Absent"
    today = datetime.now().strftime("%Y-%m-%d")
    file  = f"data/attendance_{datetime.now():%Y_%m}.csv"
    exists = os.path.exists(file)

    with open(file, "a", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=["date","emp_id","status"])
        if not exists:
            writer.writeheader()
        writer.writerow({"date":today, "emp_id":emp_id, "status":status})
    log_activity("ATTENDANCE", f"EmpID {emp_id} marked {status}")

# ── Monthly Attendance Report ──────────────────────────────────────
def monthly_attendance_report(year, month):
    file = f"data/attendance_{year}_{month:02d}.csv"
    if not os.path.exists(file):
        print("No attendance data for this month.")
        return

    summary = {}
    with open(file, "r", newline="") as f:
        for row in csv.DictReader(f):
            eid = row["emp_id"]
```

📁 Data Files Generated: data/attendance_2024_01.csv → reports/attendance_2024_01.csv →

Learning Outcomes — After Day 7, You Can:

Open & manage files

Use open() with all modes: r, w, a, x, r+, a+ confidently.

Read files 3 ways

read(), readline(), readlines() — choose the right one for each task.

Write & append data

Create new files, overwrite or append — never lose data unintentionally.

Handle file errors gracefully

try/except for FileNotFoundError, PermissionError, IOError.

Work with CSV files

csv.reader, csv.writer, DictReader, DictWriter — full CSV mastery.

Build a CSV database

Create, search, update, delete records. Filter and generate reports.

Log activities to files

Maintain persistent activity logs with timestamps using append mode.

Structure files professionally

Separate src/, data/, reports/, logs/, backups/ — industry standard.

📌 Day 7 Quick Reference Cheatsheet

File Modes

`"r"`	Read. FileNotFoundError if missing.
`"w"`	Write. ⚠️ Overwrites! Creates file.
`"a"`	Append. Adds at end. Never deletes.
`"x"`	Create. Fails if already exists.
`"r+"`	Read+Write in-place.
`"a+"`	Append+Read.

CSV Functions

`csv.reader(f)`	Row as list: row[0], row[1]...
`csv.writer(f)`	writer.writerow([...])
`csv.DictReader(f)`	Row as dict: row['name']
`csv.DictWriter(f,fields)`	writer.writerow({...})

File Handling Best Practices

- Always use `with open() as f:` (context manager)
- Always add `newline=""` when using csv module
- Use `DictReader/DictWriter` — cleaner than index
- Wrap file operations in `try/except` blocks
- Use `os.path.join()` for file paths (cross-platform)
- Check `os.path.exists()` before reading
- Keep data/, logs/, reports/ separate from src/
- Validate data BEFORE writing to CSV
- Take backups before bulk updates
- Use append mode for logs — never overwrite them

Files store your data forever. 🗄️

- 📁 File Fundamentals
- 📖 Read & Write
- ⚠️ Exception Handling
- 📊 CSV Module
- 🗄️ Data Management
- 📂 Pro Structure

Day 7 Complete ✅

Next: Object-Oriented Programming →

File Handling + CSV | Day 7 — Python Bootcamp